

Sentiment (Emotion) classification Using BERT

情緒分類使用 BERT 神經網路

Setimental (Emotional) score:

Negagive, Positive, Neutral (Objetive 客觀):

負面:0 正面:1

負面:0 正面:1 中立:2

負面:0 正面:1 中立:2 無情緒:3

Sentiment Analysis 情緒分析、情緒分類

What is Sentiment Analysis?

Where can we find the dataset?

BERT

What is BERT?

Bidirectional Encoder Representations from Transformers (BERT) is a Natural Language Processing Model proposed by Google Research in 2018. It is based on a multi-layer bidirectional Transformer, pre-trained on two unsupervised tasks using a large crossdomain corpus:

* Masked Language Modeling (MLM): 15% of the words in each sequence are replaced with a [MASK] token. The model then attempts to predict the masked words, based on the context provided by the non-masked

ones.

- * Next Sentence Prediction (NSP): the model receives pairs of sentences as input and learns to predict if the second sentence is the subsequent sentence in the original document.

BERT is deeply bidirectional, which means that it can learn the context of a word based on all the information contained in the input sequence, jointly considering previous and subsequent tokens. In fact, the use of MLM objective enables the representation to fuse the left and right contexts, allowing the pre-training of a deep bidirectional language representation model. This is a key difference comparing to previous language representation models like OpenAI GPT, which uses a unidirectional (left-to-right) language model, or ELMo, which uses a shallow concatenation of independently trained left-to-right and right-to-left language models. BERT outperformed many task-specific architectures, advancing the state of the art in a wide range of Natural Language Processing tasks, such as textual entailment, text classification and question answering.

For further details, you might want to read the original BERT paper.

什麼是 BERT ?

模型來自 Transformers 的雙向編碼器表示 (BERT) 是 Google Research 在 2018 年提出的自然語言處理模型。它基於多層雙向 Transformer，使用大型跨域語料庫對兩個無監督任務進行預訓練。

大量的訓練數據

33 億字的海量數據集

BERT 專門針對 Wikipedia (約 2.5B 字) 和 Google 的 BooksCorpus (約 8 億字) 進行了訓練。這些大型信息數據集不僅有助於 BERT 深入了解語言，還有助於了解我們的世界！

在這麼大的數據集上訓練需要很長時間。BERT 的訓練得益於新穎的 Transformer 架構，並通過使用 TPU (張量處理單元——谷歌專為大型 ML 模型構建的定制電路) 加快了訓練速度。— 64 個 TPU 在 4 天的時間裡訓練了 BERT。

- * 掩碼語言建模 (MLM) : 每個序列中 15% 的單詞被替換為一個 [MASK] 標記。然後，該模型嘗試根據非蒙面詞提供的上下文來預測蒙面詞。

- * Next Sentence Prediction (NSP) : 模型接收成對的句子作為輸入，並學習預測第二個句子是否是原始文檔中的後續句子。

BERT 是深度雙向的，這意味著它可以根據輸入序列中包含的所有信息來學習單詞的上下文，同時考慮前後標記。事實上，MLM 目標的使用使表示能夠融合左右上下文，從而允許對深度雙向語言表示模型進行預訓練。這是與以前的語言表示模型 (如 OpenAI GPT) 相比的關鍵區別，OpenAI GPT 使用單向 (從左到右) 語言模型或 ELMo，它使用獨立訓練的從左到右和從右到左語言模型的淺連接。BERT 優於許多特定於任務的架構，在廣泛的自然語言處理任務 (例如文本蘊涵、文本分類和問答) 中提升了最新技術水平。

有關更多詳細信息，您可能需要閱讀原始 BERT 論文。

source:
https://riccardo-cantini.netlify.app/post/bert_text_classification/

Transformers 變形金剛(變壓器)模型

Bert and GPT are parts of Transformer

Encoders and Decoders As mentioned, there are encoders and decoders. BERT uses encoders only, GTP uses decoders only. Both options understand language including syntax and semantics. Especially the next generation of large language models like GPT with billions of parameters do this very well.

The two models focus on different scenarios. However, since the field of foundation models is evolving, the differentiation is often fuzzier.

BERT (encoder): classification (e.g., sentiment), questions and answers, summarization, named entity recognition GPT (decoder): translation, generation (e.g., stories) The outputs of the core models are different:

BERT (encoder): Embeddings representing words with attention information in a certain context
GPT (decoder): Next words with probabilities Both models are pretrained and can be reused without intensive training. Some of them are available as open source and can be downloaded from communities like Hugging Face, others are commercial. Reuse is important, since trainings are often very resource intensive and expensive which few companies can afford.

The pretrained models can be extended and customized for different domains and specific tasks. Layers can sometimes be reused without modifications and more layers are added on top. If layers need to be modified, the new training is more expensive. The technique to customize these models is called Transfer Learning, since the same generic model can easily be transferred to other domains.

[Source](#)

Transfor Learning (遷移學習), Finetune from the pretrained model (微調一個預訓練模型)

預訓練

克漏字填空 (Mask Language Model) Masked Language Model (MLM) 句子丟進去模型進行訓練，會把一些字詞遮蓋(Mask)起來，讓模型去預測被遮蓋起來的字詞是什麼。

下文預測 (Next Sentence Prediction) Next Sentence Prediction (NSP)。預測下一個可能的句子是什麼。(為了解決文本生成，或是 QA 的任務)

微調

將一個大型的預訓練模型拿來加以運用在解決其他領域的問題上，加上自己的資料再加以微調預訓練模型，變成一個可以解決你的問題的模型，可以節省成本。

例如:將 Bert 模型施以在訓練，使用自己準備的情緒語料，變成一個情緒分類器，這就是在做微調的工作。這樣就是一種遷移學習。

單一句子分類、單一句子標註、成對句子預測，以及問答任務。

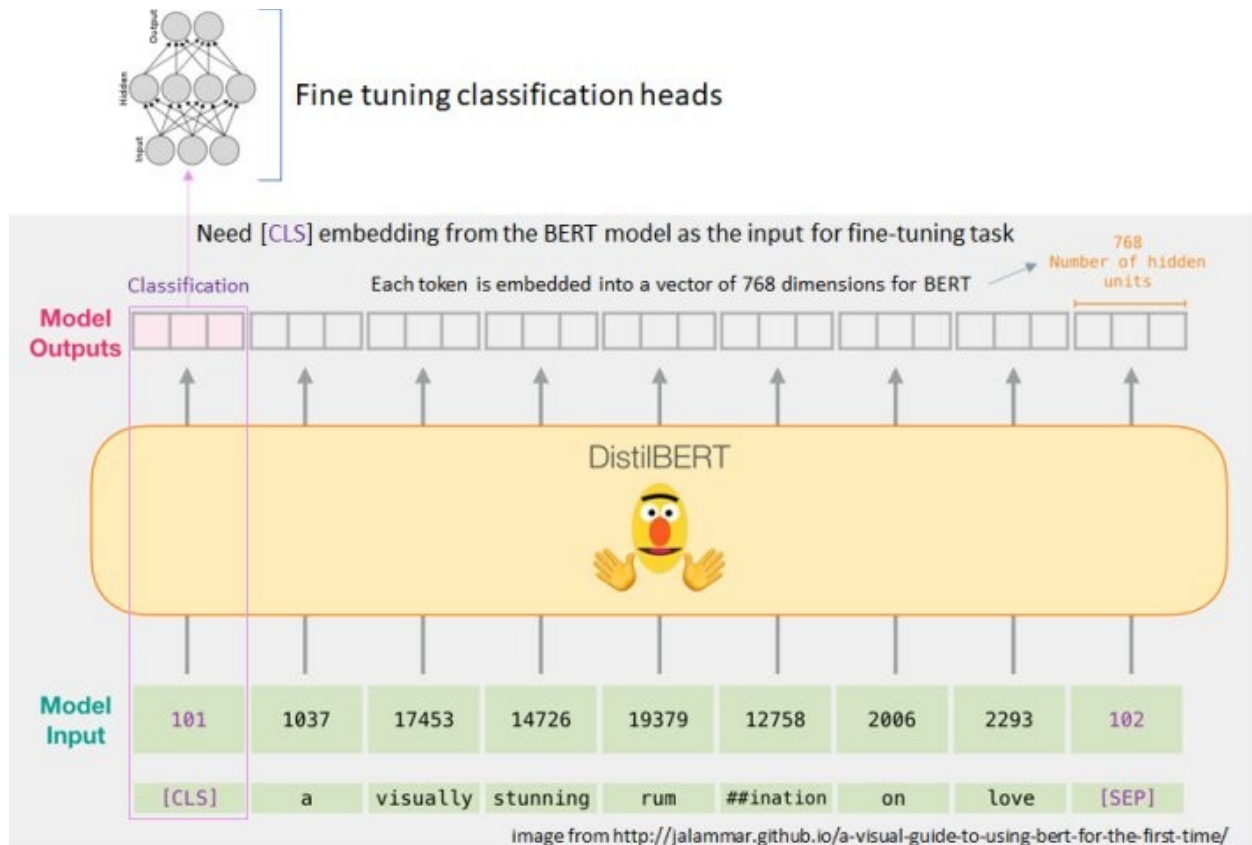
來源:台大李宏毅老師講義

BERT Input and Output (輸入與輸出)

BERT 的輸出有兩項: `last_hidden_state`, `pooler_output`

Bert input and output

Bert input and output is as follows:



Supervised Learning for text classification

A text classification procedure is composed of following components:

Training text: It is the input text through which our supervised learning model is able to learn and predict the required class.

Feature Vector: A feature vector is a vector that contains information describing the characteristics of the input data.

Labels: These are the predefined categories/classes that our model will predict.

ML Algorithm: It is the algorithm through which our model is able to deal with text classification (In our case : CNN, RNN)

Predictive Model: A model trained on the dataset can perform label predictions.

Dataset for Model Training, Validation and Testing

Slicing a single data set into three subsets.

Main steps for training machine learning models

Word Embedding

Pretrained Embedding:

https://keras-cn.readthedocs.io/en/latest/blog/word_embedding/

<http://ben.bolte.cc/blog/2016/gensim.html>

<https://gist.github.com/codekansas/15b3c2a2e9bc7a3c345138a32e029969>

<https://blog.keras.io/using-pre-trained-word-embeddings-in-a-keras-model.html>

Various Deep Learning Model

Encoder-Decoder Transformers

Attention Layer

Colab GPU

設定執行階段 變更執行階段類型 硬體加速器 選取 GPU

```
!nvidia-smi

Sun May 18 16:53:58 2025
+-----+
+-----+
| NVIDIA-SMI 550.54.15                  Driver Version: 550.54.15
| CUDA Version: 12.4                    |
+-----+-----+
+-----+
| GPU   Name                               Persistence-M | Bus-Id        Disp.A |
| Volatile Uncorr. ECC |
| Fan  Temp  Perf              Pwr:Usage/Cap |      Memory-Usage |
| GPU-Util  Compute M. |
|
| MIG M. |
+-----+-----+
=====+=====
=====|
|    0   Tesla T4                               Off | 00000000:00:04.0 Off |
| 0 |
| N/A    69C    P0              30W / 70W |      102MiB / 15360MiB |
| 0%      Default |
|
| N/A |
```

```

+-----+-----+
+-----+

```

GPU	GI	CI	PID	Type	Process name
GPU Memory					
	ID	ID			
Usage					

```

=====
=====|
+-----+-----+
+-----+

```

列出GPU 與CPU 資源

```

from tensorflow.python.client import device_lib
print(device_lib.list_local_devices())

[name: "/device:CPU:0"
 device_type: "CPU"
 memory_limit: 268435456
 locality {
 }
 incarnation: 8069296828625959847
 xla_global_id: -1
 , name: "/device:GPU:0"
 device_type: "GPU"
 memory_limit: 14619377664
 locality {
   bus_id: 1
   links {
   }
 }
 incarnation: 5437938903010203440
 physical_device_desc: "device: 0, name: Tesla T4, pci bus id:
 0000:00:04.0, compute capability: 7.5"
 xla_global_id: 416903419
]

#import os
#os.environ["CUDA_VISIBLE_DEVICES"] = "0"

```

Ubuntu 版本資訊

```
!lsb_release -a
```



```
No LSB modules are available.  
Distributor ID: Ubuntu  
Description:    Ubuntu 22.04.4 LTS  
Release:      22.04  
Codename:     jammy
```

掛載雲端硬碟

```
from google.colab import drive  
drive.mount('/content/drive')
```

Mounted at /content/drive

Insatll packages

```
transformers: For model handling.  
peft: For LoRA integration.  
datasets: For dataset handling.  
accelerate: For distributed training.  
bitsandbytes: For memory-efficient training (optional but useful  
for Qwen).  
  
#!/pip install transformers peft datasets accelerate bitsandbytes  
  
pip install evaluate  
  
Collecting evaluate  
  Downloading evaluate-0.4.3-py3-none-any.whl.metadata (9.2 kB)  
Requirement already satisfied: datasets>=2.0.0 in  
/usr/local/lib/python3.11/dist-packages (from evaluate) (2.14.4)  
Requirement already satisfied: numpy>=1.17 in  
/usr/local/lib/python3.11/dist-packages (from evaluate) (2.0.2)  
Requirement already satisfied: dill in /usr/local/lib/python3.11/dist-  
packages (from evaluate) (0.3.7)  
Requirement already satisfied: pandas in  
/usr/local/lib/python3.11/dist-packages (from evaluate) (2.2.2)  
Requirement already satisfied: requests>=2.19.0 in  
/usr/local/lib/python3.11/dist-packages (from evaluate) (2.32.3)  
Requirement already satisfied: tqdm>=4.62.1 in  
/usr/local/lib/python3.11/dist-packages (from evaluate) (4.67.1)  
Requirement already satisfied: xxhash in  
/usr/local/lib/python3.11/dist-packages (from evaluate) (3.5.0)  
Requirement already satisfied: multiprocessing in  
/usr/local/lib/python3.11/dist-packages (from evaluate) (0.70.15)  
Requirement already satisfied: fsspec>=2021.05.0 in  
/usr/local/lib/python3.11/dist-packages (from fsspec[http]>=2021.05.0-  
>evaluate) (2025.3.2)
```

Requirement already satisfied: huggingface-hub>=0.7.0 in
/usr/local/lib/python3.11/dist-packages (from evaluate) (0.31.2)

Requirement already satisfied: packaging in
/usr/local/lib/python3.11/dist-packages (from evaluate) (24.2)

Requirement already satisfied: pyarrow>=8.0.0 in
/usr/local/lib/python3.11/dist-packages (from datasets>=2.0.0->evaluate) (18.1.0)

Requirement already satisfied: aiohttp in
/usr/local/lib/python3.11/dist-packages (from datasets>=2.0.0->evaluate) (3.11.15)

Requirement already satisfied: pyyaml>=5.1 in
/usr/local/lib/python3.11/dist-packages (from datasets>=2.0.0->evaluate) (6.0.2)

Requirement already satisfied: filelock in
/usr/local/lib/python3.11/dist-packages (from huggingface-hub>=0.7.0->evaluate) (3.18.0)

Requirement already satisfied: typing-extensions>=3.7.4.3 in
/usr/local/lib/python3.11/dist-packages (from huggingface-hub>=0.7.0->evaluate) (4.13.2)

Requirement already satisfied: charset-normalizer<4,>=2 in
/usr/local/lib/python3.11/dist-packages (from requests>=2.19.0->evaluate) (3.4.2)

Requirement already satisfied: idna<4,>=2.5 in
/usr/local/lib/python3.11/dist-packages (from requests>=2.19.0->evaluate) (3.10)

Requirement already satisfied: urllib3<3,>=1.21.1 in
/usr/local/lib/python3.11/dist-packages (from requests>=2.19.0->evaluate) (2.4.0)

Requirement already satisfied: certifi>=2017.4.17 in
/usr/local/lib/python3.11/dist-packages (from requests>=2.19.0->evaluate) (2025.4.26)

Requirement already satisfied: python-dateutil>=2.8.2 in
/usr/local/lib/python3.11/dist-packages (from pandas->evaluate) (2.9.0.post0)

Requirement already satisfied: pytz>=2020.1 in
/usr/local/lib/python3.11/dist-packages (from pandas->evaluate) (2025.2)

Requirement already satisfied: tzdata>=2022.7 in
/usr/local/lib/python3.11/dist-packages (from pandas->evaluate) (2025.2)

Requirement already satisfied: aiohappyeyeballs>=2.3.0 in
/usr/local/lib/python3.11/dist-packages (from aiohttp->datasets>=2.0.0->evaluate) (2.6.1)

Requirement already satisfied: aiosignal>=1.1.2 in
/usr/local/lib/python3.11/dist-packages (from aiohttp->datasets>=2.0.0->evaluate) (1.3.2)

Requirement already satisfied: attrs>=17.3.0 in
/usr/local/lib/python3.11/dist-packages (from aiohttp->datasets>=2.0.0->evaluate) (25.3.0)

```
Requirement already satisfied: frozenlist>=1.1.1 in
/usr/local/lib/python3.11/dist-packages (from aiohttp-
>datasets>=2.0.0->evaluate) (1.6.0)
Requirement already satisfied: multidict<7.0,>=4.5 in
/usr/local/lib/python3.11/dist-packages (from aiohttp-
>datasets>=2.0.0->evaluate) (6.4.3)
Requirement already satisfied: propcache>=0.2.0 in
/usr/local/lib/python3.11/dist-packages (from aiohttp-
>datasets>=2.0.0->evaluate) (0.3.1)
Requirement already satisfied: yarll<2.0,>=1.17.0 in
/usr/local/lib/python3.11/dist-packages (from aiohttp-
>datasets>=2.0.0->evaluate) (1.20.0)
Requirement already satisfied: six>=1.5 in
/usr/local/lib/python3.11/dist-packages (from python-dateutil>=2.8.2-
>pandas->evaluate) (1.17.0)
Downloading evaluate-0.4.3-py3-none-any.whl (84 kB)
84.0/84.0 kB 3.6 MB/s eta
0:00:00
```

切換工作目錄到雲端硬碟目錄下(取用資料比較方便，可以用相對路徑)

你要改成你自己雲端硬碟目錄的路徑，有複製路徑的功能可以使用，不要用鍵盤輸入！
cd 前面一行不要加上說明文字，否則colab的cd會認不得指令

```
# cd to_your_folder
```

```
cd /content/drive/MyDrive/Colab Notebooks
```

```
/content/drive/MyDrive/Colab Notebooks
```

```
pwd
```

```
{"type": "string"}
```

```
ls -l
```

```
total 14798
```

```
-rw----- 1 root root 312580 May 18 16:44 '「1-Training model using Transformers Trainer.ipynb」的副本'
```

```
-rw----- 1 root root 37969 May 18 16:47 '「2-sentiment classification model usage.ipynb」的副本'
```

```
drwx----- 2 root root 4096 May 13 02:50 best-model-v1/
```

```
drwx----- 2 root root 4096 May 13 02:35 checkpoints_v1/
```

```
-rw----- 1 root root 14071136 May 13 02:33 dataset_reviews.csv
```

```
-rw----- 1 root root 324 Oct 26 2023 Untitled0.ipynb
```

```
-rw----- 1 root root 11527 Feb 13 06:56 Untitled10.ipynb
```

```
-rw----- 1 root root 49223 Mar 9 15:14 Untitled11.ipynb
```

```
-rw----- 1 root root      4504 Mar  3 02:12 Untitled12.ipynb
-rw----- 1 root root        258 Mar  9 14:59 Untitled13.ipynb
-rw----- 1 root root     91763 Mar 10 03:23 Untitled14.ipynb
-rw----- 1 root root    38925 Mar 24 01:59 Untitled15.ipynb
-rw----- 1 root root   40608 Mar 24 12:31 Untitled16.ipynb
-rw----- 1 root root      495 Apr 28 01:40 Untitled17.ipynb
-rw----- 1 root root   48418 Apr 28 02:27 Untitled18.ipynb
-rw----- 1 root root      324 Oct 26  2023 Untitled1.ipynb
-rw----- 1 root root   257049 Jan  4  2024 Untitled2.ipynb
-rw----- 1 root root   14151 Jul 31  2024 Untitled3.ipynb
-rw----- 1 root root    2974 Sep  3  2024 Untitled4.ipynb
-rw----- 1 root root    2951 Aug 13  2024 Untitled5.ipynb
-rw----- 1 root root     816 Sep 25  2024 Untitled6.ipynb
-rw----- 1 root root     675 Oct 11  2024 Untitled7.ipynb
-rw----- 1 root root     324 Jan 10 02:02 Untitled8.ipynb
-rw----- 1 root root    9601 Feb 20 05:57 Untitled9.ipynb
-rw----- 1 root root    8659 Sep 15  2023 初版.ipynb
-rw----- 1 root root  133783 Mar 24 02:19 '「歡迎使用 Colaboratory」的副本'
```

```
import numpy as np
import pandas as pd

# Load Huggingface transformers
from transformers import TrainingArguments, Trainer
from transformers import BertTokenizer, BertTokenizerFast,
BertForSequenceClassification
import torch
import evaluate
import datasets
```

GPU Device

```
# Setting device on GPU if available, else CPU
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print('Using device:', device)
```

Using device: cuda

Read data

簡體中文資料，資料集來自於網路，轉換成繁體中文

```
df = pd.read_csv('./dataset_reviews.csv', sep='|')
df
```

```
{
  "summary": {
    "name": "df",
    "rows": 80442,
    "fields": [
      {
        "column": "text",
        "properties": {
          "dtype": "string",
          "num_unique_values": 77059,
          "samples": [
            "?\\u597d\\u8a55\\uff01?\\u597d?\\u597d\\u8a55??\\u597d?\\u597d\\u8a55?\\u597d\\u8a55??\\u597d?\\u597d\\u8a55??\\u597d\\u8a55?\\u597d\\u8a55??\\u597d?\\u597d\\u8a55??\\u597d?\\u597d\\u8a55\\uff01",
            "\\u5abd\\u7684\\uff0c\\u8cb7\\u5c0f\\u4e86\\u9000\\u56de\\u53bb\\u63db\\u90fd\\u4e00\\u500b\\u6708\\u591a\\u4e86\\u9084\\u6c92\\u6709\\u8a18\\u56de\\u4f86\\u7d66\\u6211\\uff0c\\u60f3\\u9000\\u53c8\\u9000\\u4e0d\\u4e86\\uff0c***\\u4e86\\uff0c\\u7576\\u8cb7\\u500b\\u6559\\u8a13\\u5427\\uff0c\\u4ee5\\u5f8c\\u90fd\\u4e0d\\u6703\\u8cb7\\u9019\\u500b\\u724c\\u5b50\\u7684\\u8863\\u670d\\uff0c\\u6700\\u4e0d\\u958b\\u5fc3\\u7684\\u4e00\\u6b21\\u8cfc\\u7269\\uff0c\\u5dee\\u8a55\\u5dee\\u8a55\\u5dee\\u8a55\\u5dee\\u8a55\\u7d55\\u5c0d\\u7684\\u5dee\\u8a55",
            "\\u8932\\u5b50\\u5f88\\u597d\\u770b\\u5f88\\u5408\\u8eab\\uff0c\\u54a8\\u8a62\\u6642\\u5e97\\u4e3b\\u4e5f\\u5f88\\u71b1\\u5fc3\\uff0c\\u5f88\\u4e0d\\u932f\\u7684\\u4e00\\u6b21\\u8cfc\\u7269\\uff0c\\u4e94\\u661f"
          ],
          "semantic_type": "\"\"",
          "description": "\"\"",
          "label": "\"\"",
          "properties": {
            "dtype": "number",
            "std": 0,
            "min": 0,
            "max": 1,
            "num_unique_values": 2,
            "samples": [
              0
            ],
            "semantic_type": "\"\"",
            "description": "\"\"",
            "label": "\"\"",
            "properties": {}
          ],
          "type": "dataframe",
          "variable_name": "df"
        }
      }
    ]
  }
}
```

```
df.shape
```

```
(80442, 2)
```

```
df.groupby('label').size()
```

```
label
```

```
0    41879
```

```
1    38563
```

```
dtype: int64
```

```
df.dtypes
```

```
text    object
```

```
label    int64
```

```
dtype: object
```

Convert the format of y

Convert the format of y from int to LongTensor

Convert label using one-hot representation 輸出資料格式 one-hot 轉換

轉成用 2 個節點表達兩類

類別 0: [1 0]

類別 1: [0 1]

如果是 3 個類別用 3 個節點表之:

類別 0: [1 0 0]

類別 1: [0 1 0]

類別 2: [0 0 1]

負面情緒 Negative 0 --> [1 0]

正面情緒 Positive 1 --> [0 1]

Easy Representation 0,1,2... (內部會自動轉換為 one-hot)

負面情緒 Negative 0 --> [0]

正面情緒 Positive 1 --> [1]

如果是 3 個類別

中立情緒 Neutral 2 --> [2]

Convert category name ('正面','負面') into number (0,1)

label <--> id

```
# Map labels to integers
categories=['負面', '正面']

label_to_id = { cate : i for i, cate in enumerate(categories)}
label_to_id
{'負面': 0, '正面': 1}

id_to_label = { i : cate for i, cate in enumerate(categories)}
id_to_label
{0: '負面', 1: '正面'}
```

```
# df = df.sample(20000)
```

```
df.shape
```

 $(80442, 2)$

Conver pandas dataframe to Huggingface Dataset

```
dataset = datasets.Dataset.from_pandas(df, preserve_index=False)
```

dataset

```
Dataset({
  features: ['text', 'label'],
  num_rows: 80442
})
```

dataset[0]

```
{ 'text': '做父母一定要有劉墉這樣的心態，不斷地學習，不斷地進步，不斷地給自己補充新鮮血液，讓自己保持一顆年輕的心。我想，這是他能很好的和孩子溝通的一個重要因素。讀劉墉的文章，總能讓我看到一個快樂的平易近人的父親，他始終站在和孩子同樣的高度，給孩子創造著一個充滿愛和自由的生活環境。很喜歡劉墉在字裡行間流露出的做父母的那種小狡黠，讓人總是忍俊不禁，父母和子女之間有時候也是一種戰鬥，武力爭鬥過於低級了，智力較量才更有趣味。所以，做父母的得加把勁了，老思想老觀念注定會一敗塗地，生命不息，學習不止。家庭教育，真的是樂在其中。',  
  'label': 1}
```

```
dataset.to_pandas().head(5)
```

```
{
  "summary": {
    "name": "dataset",
    "rows": 5,
    "fields": [
      {
        "column": "text",
        "properties": {
          "dtype": "string",
          "num_unique_values": 5,
          "samples": [
            "\u0046\u005c\u0005\u0077\u0067\u0082\u0057\u004e\u00b4\u008b\u0076\u0098\u0068\u00ff\u0063\u0051\u009c\u0001\u0090\u0084\u008a\u008f\u008a\u00b4\u00ff\u0051\u007b\u0067\u004e\u005c\u0072\u0074\u005b\u0077\u0077\u0089\u00e3\u004e\u006d\u00ff\u004f\u0066\u004e\u0071\u0080\u0061\u0053\u0072\u0052\u0077\u0068\u0070\u006b\u0082\u0030\u0065\u0072\u0066\u0076\u0081\u0098\u0072\u009e\u00ff\u005f\u0075\u0066\u006f\u005b\u0065\u0051\u0053\u005e\u004e\u00ff\u006d\u0088\u0076\u008a\u007b\u006a\u005f\u008a\u0098\u005f\u0051\u0065\u0072\u0065\u0078\u0030\u0072\u0074\u005b\u0078\u0030\u005b\u005b\u005b\u0050\u005a\u005f\u0089\u0076\u0092\u0058\u0020\u0014\u0009\u004e\u0051\u005b\u004f\u0064\u004e\u0059\u0027"
          ]
        }
      }
    ]
  }
}
```

```
u90e8\\u5206\\u7bc7\\u5e45\\uff0c\\u6700\\u5f8c\\u56de\\u5230\\u95dc\\
u9375\\u554f\\u984c\\uff1a\\u96fb\\u8166\\u80fd\\u4e0d\\u80fd\\u4ee3\\
u66ff\\u4eba\\u8166\\u3002\\u548c\\u73fe\\u5728\\u6d41\\u884c\\u7684\\
u89c0\\u9ede\\u76f8\\u53cd\\uff0c\\u4f5c\\u8005\\u8a8d\\u70ba\\u4eba\\
u7684\\u67d0\\u7a2e\\u300c\\u6d1e\\u5bdf\\u300d\\u662f\\u4e0d\\u80fd\\
u88ab\\u7b97\\u6cd5\\u6a21\\u64ec\\u7684\\u3002\\u4e5f\\u8a31\\u4f5c\\
u8005\\u60f3\\u8aaa\\uff0c\\u4eba\\u7684\\u9748\\u9b42\\u662f\\u7121\\
u53ef\\u53d6\\u4ee3\\u7684\\u3002\\",\\n                "\\u4f5c\\u8005\\
u5728\\u5c11\\u5e74\\u6642\\u5373\\u559c\\u95b1\\u8b80\\uff0c\\u80fd\\
u770b\\u51fa\\u4ed6\\u7cbe\\u8b80\\u4e86\\u7121\\u6578\\u7d93\\u5178\\
uff0c\\u56e0\\u800c\\u4ed6\\u6709\\u4e00\\u500b\\u9f90\\u5927\\u7684\\
u5167\\u5fc3\\u4e16\\u754c\\u3002\\u4ed6\\u7684\\u4f5c\\u54c1\\u6700\\
u96e3\\u80fd\\u53ef\\u8cb4\\u7684\\u6709\\u5169\\u9ede\\uff0c\\u4e00\\
u662f\\u4ed6\\u7684\\u7406\\u79d1\\u77e5\\u8b58\\u4e0d\\u932f\\uff0c\\
u96d6\\u4e0d\\u80fd\\u5ab2\\u53ca\\u7f85\\u7d20\\uff0c\\u4f46\\u8207\\
u7406\\u79d1\\u77e5\\u8b58\\u5f88\\u5dee\\u7684\\u4f5c\\u5bb6\\u76f8\\
u6bd4\\uff0c\\u4ed6\\u7684\\u6587\\u7ae0\\u53ef\\u8b80\\u6027\\u8981\\
u5f37\\uff1b\\u5176\\u4e8c\\u662f\\u4ed6\\u4eba\\u683c\\u548c\\u6587\\
u98a8\\u7684\\u6a38\\u5be6\\uff0c\\u4e0d\\u9020\\u4f5c\\uff0c\\u4e0d\\
u8cb7\\u5f04\\uff0c\\u8b93\\u4eba\\u559c\\u6b61\\u3002\\u8b80\\u4ed6\\
u7684\\u4f5c\\u54c1\\uff0c\\u7336\\u5982\\u807d\\u4e00\\u500b\\u597d\\
u53cb\\u548c\\u4f60\\u8ac7\\u5fc3\\uff0c\\u5e38\\u5e38\\u559a\\u8d77\\
u5fc3\\u4e2d\\u7684\\u5f37\\u70c8\\u7684\\u5171\\u9cf4\\u3002\\u4ed6\\
u7684\\u4f5c\\u54c190\\u5e74\\u5f8c\\u7684\\u66f4\\u597d\\u4e9b\\
u3002\\u8877\\u5fc3\\u795d\\u9858\\u5468\\u570b\\u5e73\\u5065\\u5eb7\\
u5feb\\u6a02\\uff0c\\u70ba\\u4e16\\u4eba\\u5beb\\u51fa\\u66f4\\u591a\\
u597d\\u4f5c\\u54c1\\u3002\\",\\n                "\\u4f5c\\u8005\\u9577\\
u7bc7\\u5927\\u8ad6\\u501f\\u7528\\u8a73\\u7d30\\u5831\\u544a\\u6578\\
u64da\\u8655\\u7406\\u5de5\\u4f5c\\u548c\\u8a08\\u7b97\\u7d50\\u679c\\
u652f\\u6301\\u5176\\u65b0\\u89c0\\u9ede\\u3002\\u70ba\\u4ec0\\u9ebc\\
u8377\\u862d\\u66fe\\u7d93\\u7e23\\u6709\\u6b50\\u6d32\\u6700\\u9ad8\\
u7684\\u751f\\u7522\\u7387\\uff1f\\u70ba\\u4ec0\\u9ebc\\u5728\\u6587\\
u5316\\u4e0a\\u6709\\u8457\\u6df1\\u523b\\u7d10\\u5e36\\u95dc\\u4fc2\\
u7684\\u4e2d\\u570b\\u548c\\u65e5\\u672c\\u537b\\u5728\\u7d93\\u6fdf\\
u767c\\u5c55\\u4e0a\\u6709\\u8457\\u6975\\u5927\\u7684\\u5dee\\u7570\\
uff1f\\u70ba\\u4ec0\\u9ebc\\u82f1\\u570b\\u7684\\u5317\\u7f8e\\u6b96\\
u6c11\\u5730\\u9020\\u5c31\\u4e86\\u7d93\\u6fdf\\u5f37\\u5927\\u7684\\
u7f8e\\u570b\\uff0c\\u800c\\u897f\\u73ed\\u7259\\u7684\\u5317\\u7f8e\\
u6b96\\u6c11\\u537b\\u9020\\u5c31\\u4e86\\u8303\\u5f8c\\u7684\\u58a8\\
u897f\\u54e5\\uff1f\\u2026\\u2026\\u5f88\\u6709\\u50f9\\u503c\\uff0c\\
u4f46\\u4e0d\\u5305\\u62ec\\u3010\\u4e2d\\u570b\\u8fd1\\u4ee3\\u53f2\\
u5c08\\u696d\\u3011\\u3002\\\"\\n                ],\\n                \"semantic_type\":
\\\"\\\",\\n                \"description\": \\\"\\\"\\n                }\\n                },\\n                {\\n
\\\"column\": \\\"label\\\",\\n                \"properties\": {\\n                \"dtype\":
\\\"number\\\",\\n                \"std\": 0,\\n                \"min\": 1,\\n
\\\"max\": 1,\\n                \"num_unique_values\": 1,\\n                \"samples\":
[\\n                1\\n                ],\\n                \"semantic_type\": \\\"\\\",\\n
\\\"description\": \\\"\\\"\\n                }\\n                }\\n                ]\\n}\"},\"type\":\"dataframe\"}
```


Load tokenizer

Purpose: 對輸入文字進行數字代號編碼

```
[('的', 1),  
 ('了', 2),  
 ('是', 3),  
 ('不', 4),  
 ('很', 5),  
 ('我', 6),  
 ('一', 7),  
 ('個', 8),  
 ('好', 9),  
 ('就', 10),  
 ('也', 11),  
 ('這', 12),  
 ('有', 13),  
 ('還', 14),  
 ('都', 15),  
 ('酒店', 16),  
 ('不錯', 17),  
 ('在', 18),  
 ('買', 19),  
 ('次', 20)]
```

```
# Name of the BERT model
```

```
model_name = 'bert-base-chinese'
```

```
tokenizer = BertTokenizerFast.from_pretrained(model_name)
```

```
/usr/local/lib/python3.11/dist-packages/huggingface_hub/utils/  
_auth.py:94: UserWarning:
```

```
The secret `HF_TOKEN` does not exist in your Colab secrets.
```

```
To authenticate with the Hugging Face Hub, create a token in your  
settings tab (https://huggingface.co/settings/tokens), set it as  
secret in your Google Colab and restart your session.
```

```
You will be able to reuse this secret in all of your notebooks.
```

```
Please note that authentication is recommended but still optional to  
access public models or datasets.
```

```
warnings.warn(
```

```
{"model_id": "83fcb60714f54fb1a860d4d7742bb2c2", "version_major": 2, "vers  
ion_minor": 0}
```

```
{"model_id": "074ef55d0d35476891269787946fba0b", "version_major": 2, "vers  
ion_minor": 0}
```

```
{"model_id": "d7eacd6ae6c14eba978a257616b504ad", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "5b27c3c240fe481f87a40e506be237ba", "version_major": 2, "version_minor": 0}
```

```
tokenizer('你好嗎?')
```

```
{'input_ids': [101, 872, 1962, 1621, 136, 102], 'token_type_ids': [0, 0, 0, 0, 0], 'attention_mask': [1, 1, 1, 1, 1, 1]}
```

```
tokenizer.decode([101, 872, 1962, 1621, 136, 102])
```

```
{"type": "string"}
```

```
tokenizer.get_vocab()['[CLS]']
```

```
101
```

```
tokenizer.get_vocab()['你']
```

```
872
```

```
tokenizer.get_vocab()['好']
```

```
1962
```

```
tokenizer.get_vocab()['?']
```

```
136
```

```
tokenizer.get_vocab()
```

```
{'##くれる': 11241,  
'##懼': 17311,  
'##aft': 12106,  
'pad': 10257,  
'生': 4495,  
'览': 6229,  
'座': 2429,  
'##柒': 16437,  
'##cture': 11235,  
'##斐': 16213,  
'##外': 14969,  
'よ': 576,  
'##仪': 13868,  
'移': 4919,  
'##入': 14114,  
'816': 10937,  
'插': 2991,  
'骸': 7760,  
'関': 7283,
```

'##讀': 19421,
'尉': 2202,
'##ful': 10291,
'按': 2902,
'触': 6239,
'砸': 4790,
'まで': 9642,
'模': 3563,
'~': 8080,
'澳': 4078,
'窩': 4973,
'##ica': 9156,
'##ua': 9478,
'87': 8467,
'##擻': 16157,
'##-': 21069,
'##森': 16982,
'bh': 11030,
'蜓': 6052,
'误': 6428,
'##楮': 16569,
'螢': 6086,
'##天': 13465,
'##黃': 20998,
'##47': 9050,
'##点': 17214,
'性': 2595,
'研': 4777,
'##蹣': 19750,
'##fer': 10967,
'胺': 5542,
'院': 7368,
'##兆': 14099,
'##んて': 12098,
'##雖': 20483,
'##依': 13963,
'琥': 4424,
'##谐': 19512,
'入': 326,
'##琶': 17492,
'罔': 5382,
'##黻': 18043,
'隔': 7392,
'贻': 6591,
'##mah': 10394,
'esc': 9307,
'png': 10246,
'##页': 20609,
'讷': 6372,

'##儼': 14092,
'穗': 4945,
'辭': 6798,
'购': 6579,
'##2014': 8951,
'##愤': 15756,
'##東': 16403,
'##噓': 14713,
'201': 9250,
'干': 2397,
'##丕': 13742,
'励': 1225,
'金': 7032,
'##os': 8470,
'leonardo': 8579,
'署': 5392,
'72': 8325,
'mountain': 12482,
'##ω': 10506,
'##译': 19463,
'wing': 12349,
'##萄': 18900,
'嬉': 2080,
'##袞': 13838,
'##sl': 11066,
'ebay': 8886,
'pos': 9854,
'##蝇': 19123,
'307': 10486,
'##潰': 17118,
'##箴': 18096,
'>': 508,
'276': 11173,
'众': 830,
'瓢': 4479,
'殒': 3656,
'mike': 10021,
'sir': 10171,
'穗': 4950,
'##370': 13121,
'鯽': 7811,
'梓': 3452,
'碰': 4821,
'霁': 7436,
'滲': 4016,
'##鯢': 20895,
'july': 8909,
'刹': 1176,
'廟': 2448,

'龺': 7989,
'##席': 15432,
'##廬': 15509,
'##扳': 15876,
'翳': 5434,
'##実': 15199,
'##杵': 16405,
'磬': 4786,
'##拄': 15913,
'栏': 3408,
'稀': 4921,
'辉': 6778,
'峙': 2283,
'##鹈': 20964,
'舫': 5662,
'徽': 7957,
'##婆': 15095,
'颊': 7570,
'13': 12911,
'42': 8239,
'哟': 1576,
'pchome': 9780,
'##桜': 16493,
'##奥': 18697,
'庸': 2435,
'##纬': 18338,
'##懶': 15811,
'堑': 1833,
'意': 2692,
'ぬ': 559,
'擷': 3098,
'鲛': 7830,
'##6': 13383,
'##疵': 17617,
'##珪': 17464,
'hitachi': 12294,
'##ノ': 13687,
'症': 4568,
'眉': 4691,
'##潜': 17109,
'mo': 11303,
'##釐': 20088,
'##睹': 17783,
'##抵': 15911,
'羸': 5415,
'##婚': 15099,
'##蔡': 18975,
'##诉': 19458,

'##进': 19895,
'₃': 357,
'villa': 10806,
'##緣': 18282,
'<': 398,
'ww': 13279,
'捏': 2934,
'枇': 3355,
'焗': 4187,
'##祀': 17910,
'𪗇': 6487,
'v4': 10942,
'喉': 1590,
'address': 13093,
'される': 12033,
'##core': 11921,
'##屬': 15310,
'555': 11723,
'bing': 12960,
'##近': 19875,
'独': 4324,
'##繇': 18306,
'##蒲': 18948,
'paypal': 8657,
'today': 11262,
'##=': 13586,
'##弑': 15524,
'hvp': 11067,
'菲': 5838,
'跃': 6645,
'##ン': 8489,
'##滕': 17058,
'爾': 4273,
'蕴': 5943,
'##才': 11238,
'##ose': 10936,
'騙': 7700,
'##芫': 18754,
'醛': 7010,
'剧': 1196,
'藐': 5967,
'##lam': 12953,
'>': 399,
'##最': 16354,
'##酷': 20056,
'痲': 4589,
'##癱': 17668,
'##篡': 18127,
'荼': 5795,

'遂': 6876,
'corporation': 8831,
'3000': 8283,
'##弭': 15538,
'##润': 16940,
'opera': 10917,
'##找': 15880,
'藓': 5968,
'哥': 1520,
'构': 3354,
'##莠': 18871,
'鲁': 7826,
'##als': 12707,
'冖': 321,
'qs': 11974,
'##只': 14429,
'鑰': 7145,
'ならワークケートへ': 12154,
'##攬': 16174,
'彰': 2511,
'嚟': 1701,
'##邑': 19980,
'april': 9557,
'拒': 2867,
'旻': 3201,
'甥': 4498,
'can': 9109,
'mb': 9499,
'garden': 9794,
'祛': 4865,
'##吾': 14491,
'##党': 14111,
'##』': 13664,
'##惋': 15719,
'##扮': 15872,
'##漳': 17097,
'##dical': 13168,
'fgo': 11401,
'o': 222,
'塌': 1847,
'##霁': 20493,
'嚙': 1711,
'認': 6291,
'##质': 19631,
'するとあなたにもっとマッチした': 12153,
'港': 3949,
'掠': 2966,
'##區': 14338,
'溢': 3980,

'##攢': 16168,
'whatsapp': 10250,
'黎': 7944,
'##氢': 16762,
'angela': 12822,
'##瓣': 17537,
'##铉': 20251,
'##鞑': 20550,
'穫': 4953,
'usb': 8335,
'酬': 6992,
'##鄞': 20026,
'##筒': 18087,
'f5': 12173,
'锢': 7235,
'##拷': 15950,
'##荀': 18824,
'しの': 10270,
'張': 2484,
'##热': 17235,
'##牡': 17342,
'##複': 19242,
'豐': 6490,
'##said': 8944,
'##霍': 20509,
'套': 1947,
'##醞': 20063,
'##铂': 20246,
'樞': 3561,
'胃': 5518,
'530': 10830,
'##dd': 9879,
'##濃': 17140,
'##沢': 16824,
'隍': 7388,
'##走': 19681,
'饋': 7637,
'##ule': 10724,
'##锚': 20289,
'##壩': 14921,
'##怒': 15641,
'##邻': 20000,
'敗': 3134,
'##玳': 17444,
'##駿': 20756,
'##胸': 16360,
'##舜': 18715,
'##翰': 18489,
'##肽': 18569,

'囧': 1735,
'曝': 3284,
'##檣': 16648,
'##攏': 16160,
'##祇': 17913,
'銷': 7077,
'##躺': 19777,
'airmail': 11837,
'てきちゃいます': 12136,
'##割': 14257,
'##ht': 12160,
'婁': 2037,
'褲': 6194,
'唧': 1543,
'##璞': 17525,
'##干': 15454,
'in': 8217,
'晔': 1516,
'染': 3381,
'##造': 19920,
'莲': 5813,
'籍': 5093,
'##因': 14785,
'##退': 19899,
'寮': 2185,
'恥': 2615,
'##籬': 18156,
'##專': 15258,
'蔘': 5916,
'netscape': 8691,
'到': 1168,
'##帮': 15433,
'##醋': 20062,
'{' : 169,
'Y': 212,
'伊': 823,
'娛': 2024,
'屠': 2248,
'浑': 3847,
'##夥': 14976,
'##焙': 17246,
'##kv': 11145,
'step3': 9434,
'并': 2400,
'涕': 3873,
'##咆': 14524,
'祐': 4860,
'窮': 4981,
'されています': 9604,

'美': 5401,
'翹': 5426,
'蛭': 6039,
'蟑': 6096,
'孜': 2104,
'锵': 7245,
'##嘉': 14706,
'##碼': 17883,
'story': 10076,
'##讴': 19441,
'坦': 1788,
'##槩': 16651,
'##阗': 20373,
'##計': 19300,
'凍': 1120,
'jobs': 13103,
'軋': 6757,
'つ': 554,
'监': 4664,
'##ゆ': 13675,
'##腕': 18637,
'攪': 3115,
'xbox': 9077,
'粟': 5112,
'轍': 6753,
'韜': 7503,
'##聶': 18536,
'921': 12253,
'5m': 11483,
'##慕': 15767,
'client': 10989,
'##腕': 18614,
'##浜': 16910,
'##赤': 19676,
'##钠': 20222,
'ua': 11107,
'▲topdec': 10499,
'气': 3698,
'農': 6803,
'##栾': 16478,
'##擬': 16150,
'20000': 8875,
'阙': 7336,
'ai': 8578,
'少': 2208,
'##稹': 17996,
'鎏': 7113,
'紗': 5156,
'军': 1092,

'##驚': 20929,
'##蠍': 19162,
'329': 11705,
'##ag': 10814,
'##裳': 19235,
'##覺': 19271,
'##160': 12765,
'いて': 11232,
'##藐': 19024,
'紊': 5150,
'886': 12050,
'梁': 5118,
'uniqlo': 11905,
'365': 8728,
'糗': 5132,
'齊': 7968,
'##ive': 8857,
'##們': 14004,
'##顫': 20605,
'##訓': 19303,
'##整': 19756,
'##呃': 14494,
'琛': 4422,
'焜': 4191,
'竺': 5002,
'##純': 18212,
'1885': 13124,
'铰': 7210,
'##zz': 9409,
'##購': 19611,
'衷': 6141,
'駕': 7690,
'##停': 14034,
'隻': 7407,
'蕪': 2323,
'撓': 3055,
'##產': 17553,
'換': 2940,
'single': 12148,
'積': 4948,
'##廠': 15506,
'珪': 4407,
'8gb': 11257,
'別': 1166,
'綺': 5210,
'##眸': 17761,
'墜': 1871,
'##滿': 17078,
'feb': 9694,

'v6': 11051,
'##lux': 12700,
'##高': 20827,
'##厶': 13721,
'##嘿': 14735,
'006': 12526,
'☺': 8104,
'蠹': 6111,
'trip': 12358,
'惨': 2711,
'##ル': 8492,
'color': 9916,
'残': 3655,
'渗': 3935,
'齷': 7980,
'##ved': 11667,
'##妣': 15035,
'##芸': 18769,
'##霪': 20525,
'4k': 8574,
'仏': 795,
'嗟': 1630,
'對': 2205,
'##ike': 11727,
'##撚': 16114,
'##櫃': 16659,
'cosplay': 9803,
'冰': 3718,
'諡': 6319,
'颌': 7576,
'##se': 8417,
'妍': 1970,
'##徉': 15581,
'##託': 19306,
'诛': 6412,
'*': 8011,
'##毛': 16745,
'##•': 13531,
'445': 12834,
'##赁': 19652,
'##point': 11112,
'##赫': 19679,
'##慢': 15771,
'テ': 612,
'##芻': 18771,
'##霧': 20500,
'oa': 10527,
'##憚': 15790,
'茳': 5750,

'楊': 3501,
'礙': 4844,
'蒨': 5911,
'##备': 14963,
'淫': 3915,
'##运': 19874,
'悵': 2652,
'撩': 3061,
'##琅': 17471,
'513': 13310,
'500g': 12623,
'##nc': 10507,
'冠': 2167,
'##詆': 19322,
'##cer': 10326,
'##鎔': 20171,
'##曖': 16338,
'##钝': 20219,
'狡': 4322,
'靄': 7468,
'鰲': 7845,
'##rma': 13156,
'乳': 745,
'穊': 3692,
'##儲': 14090,
'##習': 18481,
'##衄': 19175,
'##閑': 20334,
'##坏': 14833,
'##灝': 17179,
'板': 3352,
'花': 5709,
'##tan': 10105,
'van': 9933,
'##楸': 16572,
'##詞': 19327,
'##彤': 15559,
'##ak': 9896,
'##赴': 19683,
'庚': 2423,
'##貪': 19633,
'##撩': 16118,
'##攬': 16173,
'主': 712,
'裂': 6162,
'##': 13603,
'橢': 3495,
'麟': 7929,
'綜': 5198,

'祝': 4867,
'##豊': 19547,
'##謝': 19399,
'##縉': 18415,
'_k': 333,
'v2': 9636,
'##10': 13117,
'っている': 11859,
'箝': 5051,
'楚': 3504,
'[unused69]': 69,
'ы': 257,
'kanshu': 13236,
'д': 237,
'traction': 12600,
'##酬': 20064,
'洵': 3830,
'蔬': 5922,
'##撻': 16128,
'105': 8423,
'ktv': 8894,
'##琕': 17476,
'##韶': 20568,
'宁': 2123,
'x86': 10652,
'傷': 1003,
'瑩': 4453,
'##佇': 13910,
'##监': 17721,
'##崩': 15366,
'应': 2418,
'##氯': 16771,
'桿': 3447,
'##焗': 17244,
'悞': 2672,
'##劇': 14263,
'##姬': 15040,
'ᄒ': 281,
'##嘅': 14703,
'##瘋': 17613,
'▲topaug': 10558,
'促': 914,
'##遭': 19958,
'6000': 8412,
'瑞': 4448,
'铛': 7197,
'饞': 7651,
'肺': 5511,
'##シャンルの': 11947,

'馬': 7679,
'剑': 1175,
'竅': 4986,
'wali': 10736,
'##60': 8581,
'##鄰': 20030,
'##儋': 14083,
'##券': 14286,
'##吋': 14454,
'##鄉': 20022,
'##岷': 15335,
'##ao': 8731,
'誕': 6293,
'##my': 9009,
'##吨': 14474,
'##侍': 13949,
'碳': 4823,
'箫': 5054,
'dota': 11636,
'##fx': 13122,
'##呢': 14507,
'汨': 3741,
'##村': 16390,
'linkedin': 11369,
'稔': 4926,
'##葦': 18936,
'##啗': 14199,
'嗎': 1621,
'255': 9866,
'噓': 1694,
'4800': 12488,
'葵': 5878,
'翹': 5435,
'##采': 20080,
'##宫': 15208,
'サイトは': 11063,
'##covery': 12548,
'##ever': 11531,
'##夯': 14983,
'r': 8082,
'##嚶': 14721,
'##®': 8646,
'##沟': 16822,
'勸': 1250,
'##拓': 15925,
'阴': 7346,
'##谗': 19513,
'矜': 4758,
'扩': 2810,

'饅': 7635,
'##混': 16978,
'雋': 7418,
'鸞': 7895,
'棱': 3482,
'101': 8359,
'##|': 13580,
'##霜': 20515,
'##颯': 20657,
'霑': 7455,
'十': 1282,
'姬': 2009,
'##描': 16046,
'##汽': 16806,
'design': 9225,
'墳': 1877,
'239': 10694,
'佣': 874,
'amd': 8992,
'汞': 3735,
'##嗑': 14679,
'...': 403,
'##uce': 12051,
'讠': 6377,
'##捧': 15999,
'##輓': 19836,
'##杭': 16400,
'##宛': 15195,
'##■': 13607,
'茗': 5751,
'鶯': 7909,
'##痠': 17651,
'##谓': 19515,
'ひ': 563,
'##ul': 10086,
'##忻': 15631,
'##銓': 20132,
'漠': 4030,
'肥': 5503,
'##芥': 18838,
'west': 8520,
'##係': 13970,
'mcu': 10738,
'淹': 3922,
'74': 8473,
'##熊': 17277,
'##闰': 20375,
'##嚮': 14769,
'ted': 11398,

'##(5)': 13570,
'##鎮': 20309,
'##i': 13519,
'way': 10590,
'##更': 16348,
'ア': 588,
'馭': 7717,
'marketing': 13189,
'##ics': 9034,
'窩': 4979,
'聶': 5479,
'鎗': 7116,
'鐙': 7263,
'集': 7415,
'##ound': 11477,
'##η': 13366,
'15058': 12347,
'##觀': 19277,
'嚙': 1710,
'働': 1007,
'##ㄅ': 21111,
'尽': 2226,
'##帥': 15428,
'辜': 6790,
'##籌': 18149,
'##噌': 14737,
'崧': 2308,
'x4': 12124,
'園': 1758,
'coach': 10784,
'σ': 487,
'##峰': 15349,
'诘': 6409,
'##尫': 15274,
'全': 1059,
'雞': 7430,
'##宕': 15190,
'##緣': 18290,
'笹': 5019,
'##欽': 20228,
'studio': 8318,
'##臀': 18675,
'433': 13131,
'ettoday': 9485,
'##何': 13919,
'佔': 861,
'##900': 10589,
'##濾': 17066,
'民': 3696,

'##蜴': 19119,
'32gb': 11155,
'##灿': 17193,
'np': 11203,
'##lv': 9084,
'璎': 4465,
'##巷': 15407,
'##诤': 19477,
'##coin': 10160,
'诿': 6431,
'##懿': 15817,
'cc': 8485,
'柜': 3385,
'润': 3883,
'##惭': 15734,
'eia': 11981,
'喬': 1605,
'lo': 12264,
'遞': 6894,
'##幫': 15453,
'桀': 3422,
'##阅': 20382,
'瑤': 4450,
'navigation': 12232,
'##違': 19946,
'##鞏': 17785,
'##bat': 12671,
'##兔': 14109,
'##戡': 15834,
'貓': 6506,
'o2o': 8590,
'勾': 1256,
'##蜂': 19101,
'##祺': 17935,
'##返': 19876,
'k': 153,
'##д': 13405,
'##冥': 14154,
'ming': 11388,
'吗': 1408,
'bob': 10542,
'舂': 5642,
'荟': 5780,
'##ora': 11455,
'##梁': 16505,
'享': 2103,
'刪': 1165,
'废': 2426,
'操': 3082,

'驻': 7728,
'##昨': 16276,
'##鸚': 20969,
'cpu': 8476,
'##sion': 9224,
'遺': 6909,
'##客': 15202,
'f': 173,
'##13433': 'س',
'##サ': 13685,
'##富': 14155,
'##鳴': 20912,
'network': 10339,
'殭': 3663,
'煮': 4215,
'睺': 4716,
'##仆': 13846,
'适': 6844,
'359': 12027,
'##5s': 13201,
'##再': 14143,
'##鷗': 20932,
'': 112,
'##tf': 11359,
'麵': 7934,
'醋': 7005,
'##ユ-': 12778,
'喱': 1608,
'##reen': 10902,
'mmorpg': 13274,
'系': 5143,
'##map': 10909,
'濫': 4010,
'##蚝': 19073,
'##ick': 9873,
'##懼': 15815,
'捆': 2928,
'##壞': 14946,
'##敘': 16192,
'##ray': 10009,
'顰': 7589,
'♣': 490,
'##恼': 15687,
'isp': 12036,
'[unused11]': 11,
'燥': 4246,
'##还': 19877,
'殞': 3664,
'潺': 4064,

'畴': 4533,
'績': 5245,
' ': 8038,
'##隊': 20443,
'鋪': 7083,
'閏': 7318,
'##涛': 16932,
'git': 10807,
'doc': 9656,
'##诊': 19459,
'a': 233,
'##琮': 17486,
'##愍': 15748,
'##阳': 20402,
'##睇': 17767,
'##ヲ': 9210,
'##諮': 19381,
'##皺': 17710,
'##ない': 8560,
'|': 384,
'##霰': 20519,
'捂': 2926,
'▲': 477,
'##ヤ': 13696,
'bank': 10717,
'gpa': 12273,
'labels': 11562,
'羸': 2088,
'##音': 20566,
'郝': 6950,
'口': 1366,
'##...': 13312,
'61': 8398,
'pacific': 12625,
'惯': 2679,
'##嗔': 14682,
'##崎': 15358,
'醬': 7016,
'##盥': 17734,
'度': 2428,
'##埡': 14865,
'##旦': 16247,
'##燃': 17291,
'##me': 8505,
'##类': 18159,
'##ヨ': 13700,
'##對': 15262,
'茴': 5761,
'盅': 4657,

'灵': 4130,
'##graphy': 12872,
'378': 12770,
'酊': 6978,
'##纖': 18327,
'1992': 8508,
'##嘖': 14626,
'盜': 4671,
'罌': 5377,
'##初': 14216,
'##多': 14971,
'##觀': 19290,
'##型': 14855,
'##閱': 20391,
'325': 10838,
'jack': 9850,
'##妝': 15023,
'叫': 1373,
'宅': 2125,
'禍': 4884,
'demo': 10828,
'創': 1201,
'琴': 4433,
'##㒼': 13467,
'笛': 4515,
'5t': 12204,
'##悍': 15693,
'##盐': 17720,
'親': 6217,
'##int': 9824,
'##琛': 17479,
'at': 8243,
'忘': 2563,
'##へ': 13693,
'##娣': 15085,
'支': 3118,
'孔': 2096,
'確': 4825,
'##房': 15848,
'顏': 7543,
'##moon': 12682,
'##www': 9718,
'##^s': 13379,
'##uy': 12574,
'##硅': 17852,
'てすか': 11244,
'##租': 17966,
'ac': 9226,
'嚙': 1675,

'##銘': 20121,
'##淨': 16896,
'激': 4080,
'豐': 6493,
'n': 8064,
'##獎': 17411,
'絕': 5179,
'冗': 1090,
'bonnie': 12972,
'踩': 6678,
'##波': 16854,
'##兩': 14117,
'##占': 14361,
'##築': 18121,
'##捱': 16003,
'拱': 2890,
'##順': 20613,
'##餵': 20690,
't3': 11291,
'籲': 5100,
'ul': 10293,
'貂': 6503,
'##堪': 16554,
'駟': 7725,
'##褥': 19248,
'弈': 2463,
'##丁': 13479,
'##弦': 15535,
'紙': 5158,
'堯': 1839,
'##ress': 11142,
'##敗': 19628,
'366': 11949,
'謔': 6338,
'##逼': 19930,
'##sson': 12582,
'醺': 7017,
'厲': 1341,
'除': 7370,
'206': 9899,
'abc': 8425,
'##畸': 17592,
'##h': 8199,
'毘': 3686,
'叁': 1345,
'##丿': 21116,
'炊': 4141,
'稻': 4940,
'##ett': 13151,

```
'##醃': 20059,  
'##畢': 17582,  
...}
```

Tokenize text

```
def tokenize_function(example):  
    return tokenizer(  
        example["text"],  
        padding="max_length",  
        max_length=512,  
        truncation=True,  
    )  
  
# Tokenize the dataset  
tokenized_dataset = dataset.map(tokenize_function, batched=True)  
  
{  
    "model_id": "eeb13019adba4929ac4340ceb1b5b208",  
    "version_major": 2,  
    "version_minor": 0  
}  
  
tokenized_dataset  
  
Dataset({  
    features: ['text', 'label', 'input_ids', 'token_type_ids',  
    'attention_mask'],  
    num_rows: 80442  
})  
  
#tokenized_dataset['input_ids'][0]
```

Split dataset for training and testing

Split dataset: Train, Test (Val)

Training set: 訓練資料集 --> 給模型讀進去訓練

Test set: 測試資料集 --> 驗證或測試模型的準確度

Split dataset into 90% for training and 10% for testing

```
tokenized_dataset = tokenized_dataset.train_test_split(test_size=0.05,  
    seed=1234)  
train_data = tokenized_dataset["train"]  
test_data = tokenized_dataset["test"]  
  
train_data
```

```

Dataset({
    features: ['text', 'label', 'input_ids', 'token_type_ids',
'attention_mask'],
    num_rows: 76419
})

print(test_data)

Dataset({
    features: ['text', 'label', 'input_ids', 'token_type_ids',
'attention_mask'],
    num_rows: 4023
})

# train_data[0]

```

Encode (Tokenize) input sentences X

```

tokenizer(
    text=list(input_sentences),
    add_special_tokens=True,
    max_length=250, # 文件若較長，必須設定數字大一些，最多 512 字
    truncation=True,
    #padding=True,
    padding='max_length',
    return_tensors='pt',
    return_token_type_ids = False,
    return_attention_mask = True,
    verbose = True)

```

Make the length of every document equal to 250

Padding(文章填塞變成長度一樣): 文章必須長度一致 最多為 250 個字詞

Load model

Load pretrained Bert model

模型載入方式

- (1) 可以自己拼接模型，但需要一些深度學習基礎
- (2) 使用 BertForSequenceClassification 類別，自動拼接一個具有多類別輸出層的分類器 classifier

```

model = BertForSequenceClassification.from_pretrained('ckiplab/albert-
base-chinese', num_labels=2) # 會自動決定多少類別

```



```
device = torch.device('cuda') if torch.cuda.is_available() else
torch.device('cpu')
device
```

```
device(type='cuda')
```

```
# Google 標準模型
```

```
#model = BertForSequenceClassification.from_pretrained('bert-base-
chinese', num_labels=2).to("cpu") # Google 標準模型: GPU 16G batch 16
```

```
# 中研院資訊所的繁體中文BERT 模型
```

```
#model =
```

```
BertForSequenceClassification.from_pretrained('ckiplab/albert-base-
chinese', num_labels=2).to(device) #Albert 標準模型
```

```
model = BertForSequenceClassification.from_pretrained('ckiplab/albert-
tiny-chinese', num_labels=2).to(device) #Albert 小模型
```

```
{"model_id": "617b8dcc69734f29a3c9270affd5c736", "version_major": 2, "vers
ion_minor": 0}
```

You are using a model of type albert to instantiate a model of type bert. This is not supported for all configurations of models and can yield errors.

```
{"model_id": "de5746e8167f486aa0fa426c731f9dd6", "version_major": 2, "vers
ion_minor": 0}
```

Some weights of BertForSequenceClassification were not initialized from the model checkpoint at ckiplab/albert-tiny-chinese and are newly initialized: ['bert.embeddings.LayerNorm.bias',

'bert.embeddings.LayerNorm.weight',
'bert.embeddings.position_embeddings.weight',
'bert.embeddings.token_type_embeddings.weight',
'bert.embeddings.word_embeddings.weight',
'bert.encoder.layer.0.attention.output.LayerNorm.bias',
'bert.encoder.layer.0.attention.output.LayerNorm.weight',
'bert.encoder.layer.0.attention.output.dense.bias',
'bert.encoder.layer.0.attention.output.dense.weight',
'bert.encoder.layer.0.attention.self.key.bias',
'bert.encoder.layer.0.attention.self.key.weight',
'bert.encoder.layer.0.attention.self.query.bias',
'bert.encoder.layer.0.attention.self.query.weight',
'bert.encoder.layer.0.attention.self.value.bias',
'bert.encoder.layer.0.attention.self.value.weight',
'bert.encoder.layer.0.intermediate.dense.bias',
'bert.encoder.layer.0.intermediate.dense.weight',
'bert.encoder.layer.0.output.LayerNorm.bias',
'bert.encoder.layer.0.output.LayerNorm.weight',
'bert.encoder.layer.0.output.dense.bias',
'bert.encoder.layer.0.output.dense.weight',
'bert.encoder.layer.1.attention.output.LayerNorm.bias',

```
'bert.encoder.layer.1.attention.output.LayerNorm.weight',
'bert.encoder.layer.1.attention.output.dense.bias',
'bert.encoder.layer.1.attention.output.dense.weight',
'bert.encoder.layer.1.attention.self.key.bias',
'bert.encoder.layer.1.attention.self.key.weight',
'bert.encoder.layer.1.attention.self.query.bias',
'bert.encoder.layer.1.attention.self.query.weight',
'bert.encoder.layer.1.attention.self.value.bias',
'bert.encoder.layer.1.attention.self.value.weight',
'bert.encoder.layer.1.intermediate.dense.bias',
'bert.encoder.layer.1.intermediate.dense.weight',
'bert.encoder.layer.1.output.LayerNorm.bias',
'bert.encoder.layer.1.output.LayerNorm.weight',
'bert.encoder.layer.1.output.dense.bias',
'bert.encoder.layer.1.output.dense.weight',
'bert.encoder.layer.2.attention.output.LayerNorm.bias',
'bert.encoder.layer.2.attention.output.LayerNorm.weight',
'bert.encoder.layer.2.attention.output.dense.bias',
'bert.encoder.layer.2.attention.output.dense.weight',
'bert.encoder.layer.2.attention.self.key.bias',
'bert.encoder.layer.2.attention.self.key.weight',
'bert.encoder.layer.2.attention.self.query.bias',
'bert.encoder.layer.2.attention.self.query.weight',
'bert.encoder.layer.2.attention.self.value.bias',
'bert.encoder.layer.2.attention.self.value.weight',
'bert.encoder.layer.2.intermediate.dense.bias',
'bert.encoder.layer.2.intermediate.dense.weight',
'bert.encoder.layer.2.output.LayerNorm.bias',
'bert.encoder.layer.2.output.LayerNorm.weight',
'bert.encoder.layer.2.output.dense.bias',
'bert.encoder.layer.2.output.dense.weight',
'bert.encoder.layer.3.attention.output.LayerNorm.bias',
'bert.encoder.layer.3.attention.output.LayerNorm.weight',
'bert.encoder.layer.3.attention.output.dense.bias',
'bert.encoder.layer.3.attention.output.dense.weight',
'bert.encoder.layer.3.attention.self.key.bias',
'bert.encoder.layer.3.attention.self.key.weight',
'bert.encoder.layer.3.attention.self.query.bias',
'bert.encoder.layer.3.attention.self.query.weight',
'bert.encoder.layer.3.attention.self.value.bias',
'bert.encoder.layer.3.attention.self.value.weight',
'bert.encoder.layer.3.intermediate.dense.bias',
'bert.encoder.layer.3.intermediate.dense.weight',
'bert.encoder.layer.3.output.LayerNorm.bias',
'bert.encoder.layer.3.output.LayerNorm.weight',
'bert.encoder.layer.3.output.dense.bias',
'bert.encoder.layer.3.output.dense.weight', 'bert.pooler.dense.bias',
'bert.pooler.dense.weight', 'classifier.bias', 'classifier.weight']
```

You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

模型每個參數層都是 Trainable

```
def print_model_details(model):
    print("\n" + "="*80)
    print("MODEL ARCHITECTURE WITH TRAINABILITY STATUS")
    print("="*80)

    trainable_params = 0
    non_trainable_params = 0

    # Print each layer with details
    for idx, (name, layer) in enumerate(model.named_modules(), 1):
        if not list(layer.named_children()): # Only print leaf nodes
            (actual layers)
            param_count = sum(p.numel() for p in layer.parameters())
            status = "TRAINABLE" if any(p.requires_grad for p in
layer.parameters()) else "NON-TRAINABLE"

            print(f"\nLayer #{idx:02d}")
            print(f"├ Name: {name}")
            print(f"├ Type: {layer.__class__.__name__}")
            print(f"├ Details: {layer}")
            print(f"├ Parameters: {param_count:,}")
            print(f"└ Status: {status}")

            if status == "TRAINABLE":
                trainable_params += param_count
            else:
                non_trainable_params += param_count

    total_params = trainable_params + non_trainable_params
    trainable_percentage = (trainable_params / total_params) * 100 if
total_params > 0 else 0

    print("\n" + "="*80)
    print(f"Total Trainable Parameters: {trainable_params:,}")
    print(f"Total Non-Trainable Parameters: {non_trainable_params:,}")
    print(f"Total Parameters: {total_params:,}")
    print(f"Trainable Parameters Percentage:
{trainable_percentage:.2f}%")
    print("="*80)

print_model_details(model)
```

```
=====
=====
```

MODEL ARCHITECTURE WITH TRAINABILITY STATUS

=====

Layer #04

- └ Name: bert.embeddings.word_embeddings
- └ Type: Embedding
- └ Details: Embedding(21128, 312, padding_idx=0)
- └ Parameters: 6,591,936
- └ Status: TRAINABLE

Layer #05

- └ Name: bert.embeddings.position_embeddings
- └ Type: Embedding
- └ Details: Embedding(512, 312)
- └ Parameters: 159,744
- └ Status: TRAINABLE

Layer #06

- └ Name: bert.embeddings.token_type_embeddings
- └ Type: Embedding
- └ Details: Embedding(2, 312)
- └ Parameters: 624
- └ Status: TRAINABLE

Layer #07

- └ Name: bert.embeddings.LayerNorm
- └ Type: LayerNorm
- └ Details: LayerNorm((312,), eps=1e-12, elementwise_affine=True)
- └ Parameters: 624
- └ Status: TRAINABLE

Layer #08

- └ Name: bert.embeddings.dropout
- └ Type: Dropout
- └ Details: Dropout(p=0.0, inplace=False)
- └ Parameters: 0
- └ Status: NON-TRAINABLE

Layer #14

- └ Name: bert.encoder.layer.0.attention.self.query
- └ Type: Linear
- └ Details: Linear(in_features=312, out_features=312, bias=True)
- └ Parameters: 97,656
- └ Status: TRAINABLE

Layer #15

- └ Name: bert.encoder.layer.0.attention.self.key
- └ Type: Linear
- └ Details: Linear(in_features=312, out_features=312, bias=True)

- Parameters: 97,656
- Status: TRAINABLE

Layer #16

- Name: bert.encoder.layer.0.attention.self.value
- Type: Linear
- Details: Linear(in_features=312, out_features=312, bias=True)
- Parameters: 97,656
- Status: TRAINABLE

Layer #17

- Name: bert.encoder.layer.0.attention.self.dropout
- Type: Dropout
- Details: Dropout(p=0.0, inplace=False)
- Parameters: 0
- Status: NON-TRAINABLE

Layer #19

- Name: bert.encoder.layer.0.attention.output.dense
- Type: Linear
- Details: Linear(in_features=312, out_features=312, bias=True)
- Parameters: 97,656
- Status: TRAINABLE

Layer #20

- Name: bert.encoder.layer.0.attention.output.LayerNorm
- Type: LayerNorm
- Details: LayerNorm((312,), eps=1e-12, elementwise_affine=True)
- Parameters: 624
- Status: TRAINABLE

Layer #21

- Name: bert.encoder.layer.0.attention.output.dropout
- Type: Dropout
- Details: Dropout(p=0.0, inplace=False)
- Parameters: 0
- Status: NON-TRAINABLE

Layer #23

- Name: bert.encoder.layer.0.intermediate.dense
- Type: Linear
- Details: Linear(in_features=312, out_features=1248, bias=True)
- Parameters: 390,624
- Status: TRAINABLE

Layer #24

- Name: bert.encoder.layer.0.intermediate.intermediate_act_fn
- Type: GELUActivation
- Details: GELUActivation()
- Parameters: 0

└─ Status: NON-TRAINABLE

Layer #26

└─ Name: bert.encoder.layer.0.output.dense
└─ Type: Linear
└─ Details: Linear(in_features=1248, out_features=312, bias=True)
└─ Parameters: 389,688
└─ Status: TRAINABLE

Layer #27

└─ Name: bert.encoder.layer.0.output.LayerNorm
└─ Type: LayerNorm
└─ Details: LayerNorm((312,), eps=1e-12, elementwise_affine=True)
└─ Parameters: 624
└─ Status: TRAINABLE

Layer #28

└─ Name: bert.encoder.layer.0.output.dropout
└─ Type: Dropout
└─ Details: Dropout(p=0.0, inplace=False)
└─ Parameters: 0
└─ Status: NON-TRAINABLE

Layer #32

└─ Name: bert.encoder.layer.1.attention.self.query
└─ Type: Linear
└─ Details: Linear(in_features=312, out_features=312, bias=True)
└─ Parameters: 97,656
└─ Status: TRAINABLE

Layer #33

└─ Name: bert.encoder.layer.1.attention.self.key
└─ Type: Linear
└─ Details: Linear(in_features=312, out_features=312, bias=True)
└─ Parameters: 97,656
└─ Status: TRAINABLE

Layer #34

└─ Name: bert.encoder.layer.1.attention.self.value
└─ Type: Linear
└─ Details: Linear(in_features=312, out_features=312, bias=True)
└─ Parameters: 97,656
└─ Status: TRAINABLE

Layer #35

└─ Name: bert.encoder.layer.1.attention.self.dropout
└─ Type: Dropout
└─ Details: Dropout(p=0.0, inplace=False)
└─ Parameters: 0
└─ Status: NON-TRAINABLE

Layer #37

- └ Name: bert.encoder.layer.1.attention.output.dense
- └ Type: Linear
- └ Details: Linear(in_features=312, out_features=312, bias=True)
- └ Parameters: 97,656
- └ Status: TRAINABLE

Layer #38

- └ Name: bert.encoder.layer.1.attention.output.LayerNorm
- └ Type: LayerNorm
- └ Details: LayerNorm((312,), eps=1e-12, elementwise_affine=True)
- └ Parameters: 624
- └ Status: TRAINABLE

Layer #39

- └ Name: bert.encoder.layer.1.attention.output.dropout
- └ Type: Dropout
- └ Details: Dropout(p=0.0, inplace=False)
- └ Parameters: 0
- └ Status: NON-TRAINABLE

Layer #41

- └ Name: bert.encoder.layer.1.intermediate.dense
- └ Type: Linear
- └ Details: Linear(in_features=312, out_features=1248, bias=True)
- └ Parameters: 390,624
- └ Status: TRAINABLE

Layer #42

- └ Name: bert.encoder.layer.1.intermediate.intermediate_act_fn
- └ Type: GELUActivation
- └ Details: GELUActivation()
- └ Parameters: 0
- └ Status: NON-TRAINABLE

Layer #44

- └ Name: bert.encoder.layer.1.output.dense
- └ Type: Linear
- └ Details: Linear(in_features=1248, out_features=312, bias=True)
- └ Parameters: 389,688
- └ Status: TRAINABLE

Layer #45

- └ Name: bert.encoder.layer.1.output.LayerNorm
- └ Type: LayerNorm
- └ Details: LayerNorm((312,), eps=1e-12, elementwise_affine=True)
- └ Parameters: 624
- └ Status: TRAINABLE

Layer #46

- └ Name: bert.encoder.layer.1.output.dropout
- └ Type: Dropout
- └ Details: Dropout(p=0.0, inplace=False)
- └ Parameters: 0
- └ Status: NON-TRAINABLE

Layer #50

- └ Name: bert.encoder.layer.2.attention.self.query
- └ Type: Linear
- └ Details: Linear(in_features=312, out_features=312, bias=True)
- └ Parameters: 97,656
- └ Status: TRAINABLE

Layer #51

- └ Name: bert.encoder.layer.2.attention.self.key
- └ Type: Linear
- └ Details: Linear(in_features=312, out_features=312, bias=True)
- └ Parameters: 97,656
- └ Status: TRAINABLE

Layer #52

- └ Name: bert.encoder.layer.2.attention.self.value
- └ Type: Linear
- └ Details: Linear(in_features=312, out_features=312, bias=True)
- └ Parameters: 97,656
- └ Status: TRAINABLE

Layer #53

- └ Name: bert.encoder.layer.2.attention.self.dropout
- └ Type: Dropout
- └ Details: Dropout(p=0.0, inplace=False)
- └ Parameters: 0
- └ Status: NON-TRAINABLE

Layer #55

- └ Name: bert.encoder.layer.2.attention.output.dense
- └ Type: Linear
- └ Details: Linear(in_features=312, out_features=312, bias=True)
- └ Parameters: 97,656
- └ Status: TRAINABLE

Layer #56

- └ Name: bert.encoder.layer.2.attention.output.LayerNorm
- └ Type: LayerNorm
- └ Details: LayerNorm((312,), eps=1e-12, elementwise_affine=True)
- └ Parameters: 624
- └ Status: TRAINABLE

Layer #57

- └ Name: bert.encoder.layer.2.attention.output.dropout
- └ Type: Dropout
- └ Details: Dropout(p=0.0, inplace=False)
- └ Parameters: 0
- └ Status: NON-TRAINABLE

Layer #59

- └ Name: bert.encoder.layer.2.intermediate.dense
- └ Type: Linear
- └ Details: Linear(in_features=312, out_features=1248, bias=True)
- └ Parameters: 390,624
- └ Status: TRAINABLE

Layer #60

- └ Name: bert.encoder.layer.2.intermediate.intermediate_act_fn
- └ Type: GELUActivation
- └ Details: GELUActivation()
- └ Parameters: 0
- └ Status: NON-TRAINABLE

Layer #62

- └ Name: bert.encoder.layer.2.output.dense
- └ Type: Linear
- └ Details: Linear(in_features=1248, out_features=312, bias=True)
- └ Parameters: 389,688
- └ Status: TRAINABLE

Layer #63

- └ Name: bert.encoder.layer.2.output.LayerNorm
- └ Type: LayerNorm
- └ Details: LayerNorm((312,), eps=1e-12, elementwise_affine=True)
- └ Parameters: 624
- └ Status: TRAINABLE

Layer #64

- └ Name: bert.encoder.layer.2.output.dropout
- └ Type: Dropout
- └ Details: Dropout(p=0.0, inplace=False)
- └ Parameters: 0
- └ Status: NON-TRAINABLE

Layer #68

- └ Name: bert.encoder.layer.3.attention.self.query
- └ Type: Linear
- └ Details: Linear(in_features=312, out_features=312, bias=True)
- └ Parameters: 97,656
- └ Status: TRAINABLE

Layer #69

- └ Name: bert.encoder.layer.3.attention.self.key

- └ Type: Linear
- └ Details: Linear(in_features=312, out_features=312, bias=True)
- └ Parameters: 97,656
- └ Status: TRAINABLE

Layer #70

- └ Name: bert.encoder.layer.3.attention.self.value
- └ Type: Linear
- └ Details: Linear(in_features=312, out_features=312, bias=True)
- └ Parameters: 97,656
- └ Status: TRAINABLE

Layer #71

- └ Name: bert.encoder.layer.3.attention.self.dropout
- └ Type: Dropout
- └ Details: Dropout(p=0.0, inplace=False)
- └ Parameters: 0
- └ Status: NON-TRAINABLE

Layer #73

- └ Name: bert.encoder.layer.3.attention.output.dense
- └ Type: Linear
- └ Details: Linear(in_features=312, out_features=312, bias=True)
- └ Parameters: 97,656
- └ Status: TRAINABLE

Layer #74

- └ Name: bert.encoder.layer.3.attention.output.LayerNorm
- └ Type: LayerNorm
- └ Details: LayerNorm((312,), eps=1e-12, elementwise_affine=True)
- └ Parameters: 624
- └ Status: TRAINABLE

Layer #75

- └ Name: bert.encoder.layer.3.attention.output.dropout
- └ Type: Dropout
- └ Details: Dropout(p=0.0, inplace=False)
- └ Parameters: 0
- └ Status: NON-TRAINABLE

Layer #77

- └ Name: bert.encoder.layer.3.intermediate.dense
- └ Type: Linear
- └ Details: Linear(in_features=312, out_features=1248, bias=True)
- └ Parameters: 390,624
- └ Status: TRAINABLE

Layer #78

- └ Name: bert.encoder.layer.3.intermediate.intermediate_act_fn
- └ Type: GELUActivation

- └ Details: GELUActivation()
- └ Parameters: 0
- └ Status: NON-TRAINABLE

Layer #80

- └ Name: bert.encoder.layer.3.output.dense
- └ Type: Linear
- └ Details: Linear(in_features=1248, out_features=312, bias=True)
- └ Parameters: 389,688
- └ Status: TRAINABLE

Layer #81

- └ Name: bert.encoder.layer.3.output.LayerNorm
- └ Type: LayerNorm
- └ Details: LayerNorm((312,), eps=1e-12, elementwise_affine=True)
- └ Parameters: 624
- └ Status: TRAINABLE

Layer #82

- └ Name: bert.encoder.layer.3.output.dropout
- └ Type: Dropout
- └ Details: Dropout(p=0.0, inplace=False)
- └ Parameters: 0
- └ Status: NON-TRAINABLE

Layer #84

- └ Name: bert.pooler.dense
- └ Type: Linear
- └ Details: Linear(in_features=312, out_features=312, bias=True)
- └ Parameters: 97,656
- └ Status: TRAINABLE

Layer #85

- └ Name: bert.pooler.activation
- └ Type: Tanh
- └ Details: Tanh()
- └ Parameters: 0
- └ Status: NON-TRAINABLE

Layer #86

- └ Name: dropout
- └ Type: Dropout
- └ Details: Dropout(p=0.0, inplace=False)
- └ Parameters: 0
- └ Status: NON-TRAINABLE

Layer #87

- └ Name: classifier
- └ Type: Linear
- └ Details: Linear(in_features=312, out_features=2, bias=True)

```
└ Parameters: 626
└ Status: TRAINABLE
```

```
=====
=====
Total Trainable Parameters: 11,539,946
Total Non-Trainable Parameters: 0
Total Parameters: 11,539,946
Trainable Parameters Percentage: 100.00%
=====
=====
```

Evaluation metrics using accuracy

```
def compute_metrics(eval_pred):
    predictions, labels = eval_pred
    predictions = np.argmax(predictions, axis=1) # Convert
    probabilities to predicted labels
    metric = evaluate.load('accuracy')
    return metric.compute(predictions=predictions, references=labels)
```

Training arguments

`save_strategy` (str or [`~trainer_utils.IntervalStrategy`], *optional*, defaults to "steps"): The checkpoint save strategy to adopt during training. Possible values are:

- `"no"`: No save is done during training.
- `"epoch"`: Save is done at the end of each epoch.
- `"steps"`: Save is done every `save_steps`.

`save_steps` (int, *optional*, defaults to 500): Number of updates steps before two checkpoint saves if `save_strategy="steps"`.

`save_total_limit` (int, *optional*): If a value is passed, will limit the total amount of checkpoints. Deletes the older checkpoints in `output_dir`.

`no_cuda` (bool, *optional*, defaults to False): Whether to not use CUDA even when it is available or not.

```
training_args = TrainingArguments(
    output_dir=repo_name,
    group_by_length=True,
    length_column_name='input_length',
    per_device_train_batch_size=24,
    gradient_accumulation_steps=2,
    evaluation_strategy="steps",
    num_train_epochs=20,
    fp16=True,
    save_steps=1000,
```

```

        save_strategy='steps', # we cannot set it to "no". Otherwise, the
model cannot guess the best checkpoint.
        eval_steps=1000,
        logging_steps=1000,
        learning_rate=5e-5,
        warmup_steps=500,
        save_total_limit=3,
        load_best_model_at_end = True # this will let the model save the
best checkpoint
    )

```

<https://stackoverflow.com/questions/62525680/save-only-best-weights-with-huggingface-transformers>

<https://discuss.huggingface.co/t/save-only-best-model-in-trainer/8442>

requires several GB of GPU memory

```

training_args = TrainingArguments(
    # 訓練多少回合與批量
    num_train_epochs=5, # Number of training epochs
    output_dir="checkpoints_v1", # Output directory for checkpoints

    learning_rate=5e-5, # Learning rate for the optimizer
    weight_decay=0.01, # Weight decay for regularization
    warmup_steps=500,
    seed=101,

    per_device_train_batch_size=64, # Batch size per device
    per_device_eval_batch_size=32, # Batch size per device for
evaluation

    eval_strategy='steps', # Evaluate after each epoch
    save_strategy="steps", # Save model checkpoints after each epoch

    #load_best_model_at_end=True, # Load the best model based on the
chosen metric
    save_total_limit=2,
    push_to_hub=False, # Disable pushing the model to the Hugging
Face Hub
    report_to="none", # Disable logging to Weight&Bias
    #fp16=True, # 是否用此精度訓練Whether to use fp16 16-bit (mixed)
precision training instead of 32-bit training.
    #bf16=True, # for 新型GPU 才能設定bf16
    logging_steps=100, # 計算準確度 損失 模型存檔
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_data,
    eval_dataset=test_data,

```

```
# callbacks=[EarlyStoppingCallback(early_stopping_pa)],
compute_metrics=compute_metrics,
)
```

Resume training from the last checkpoint if available

接續訓練

訓練後，手動載入之前訓練的分類器權重

```
#model_path='checkpoints' # 訓練後的模型存放路徑
#model = BertForSequenceClassification.from_pretrained(model_path,
num_labels=2)
```

Let's train the model

```
%%time
trainer.train()

{"model_id": "8ff1981afcf243c49e16bb26b8b6cbf4", "version_major": 2, "version_minor": 0}

<IPython.core.display.HTML object>

{"model_id": "aa09204ad6d342758039728ba1117310", "version_major": 2, "version_minor": 0}

CPU times: user 1h 14min 45s, sys: 16.8 s, total: 1h 15min 2s
Wall time: 1h 16min 22s

TrainOutput(global_step=5975, training_loss=0.25929098217058383,
metrics={'train_runtime': 4582.3346, 'train_samples_per_second':
83.384, 'train_steps_per_second': 1.304, 'total_flos':
5619714263009280.0, 'train_loss': 0.25929098217058383, 'epoch': 5.0})
```

Evaluation

```
# 最後一個模型的準確度
trainer.evaluate()

<IPython.core.display.HTML object>

{'eval_loss': 0.23522409796714783,
'eval_accuracy': 0.9082774049217002,
'eval_runtime': 14.4672,
'eval_samples_per_second': 278.078,
```

```
'eval_steps_per_second': 8.709,  
'epoch': 5.0}
```

模型之存檔與讀取檔案

評估後會存檔，並刪除前一次較差 (val_loss) 的模型，保留較佳的模型，一直會有兩個模型儲存著。

Save model

trainer.save_model()

```
# First way to save model  
# Three files are saved: config.json pytorch_model.bin  
training_args.bin  
model_path='best-model-v1'  
trainer.save_model(model_path)
```

model.save_pretrained()

```
# Second way to save the model  
# Two files are saved: config.json pytorch_model.bin  
# model_path = "best-model-v1"  
# model.save_pretrained(model_path)
```

Save tokenizer

```
model_path = "best-model-v1"  
tokenizer.save_pretrained(model_path)  
  
( 'best-model-v1/tokenizer_config.json',  
  'best-model-v1/special_tokens_map.json',  
  'best-model-v1/vocab.txt',  
  'best-model-v1/added_tokens.json',  
  'best-model-v1/tokenizer.json')
```

Load model and tokenizer

```
model_path = "best-model-v1"  
model = BertForSequenceClassification.from_pretrained(model_path,  
num_labels=2)  
model.to(device)
```

```

BertForSequenceClassification(
  (bert): BertModel(
    (embeddings): BertEmbeddings(
      (word_embeddings): Embedding(21128, 312, padding_idx=0)
      (position_embeddings): Embedding(512, 312)
      (token_type_embeddings): Embedding(2, 312)
      (LayerNorm): LayerNorm((312,)), eps=1e-12,
      elementwise_affine=True)
    (dropout): Dropout(p=0.0, inplace=False)
  )
  (encoder): BertEncoder(
    (layer): ModuleList(
      (0-3): 4 x BertLayer(
        (attention): BertAttention(
          (self): BertSdpaSelfAttention(
            (query): Linear(in_features=312, out_features=312,
bias=True)
            (key): Linear(in_features=312, out_features=312,
bias=True)
            (value): Linear(in_features=312, out_features=312,
bias=True)
            (dropout): Dropout(p=0.0, inplace=False)
          )
          (output): BertSelfOutput(
            (dense): Linear(in_features=312, out_features=312,
bias=True)
            (LayerNorm): LayerNorm((312,)), eps=1e-12,
            elementwise_affine=True)
            (dropout): Dropout(p=0.0, inplace=False)
          )
        )
        (intermediate): BertIntermediate(
          (dense): Linear(in_features=312, out_features=1248,
bias=True)
          (intermediate_act_fn): GELUActivation()
        )
        (output): BertOutput(
          (dense): Linear(in_features=1248, out_features=312,
bias=True)
          (LayerNorm): LayerNorm((312,)), eps=1e-12,
          elementwise_affine=True)
          (dropout): Dropout(p=0.0, inplace=False)
        )
      )
    )
  )
  (pooler): BertPooler(
    (dense): Linear(in_features=312, out_features=312, bias=True)
    (activation): Tanh()
  )
)

```



```

)
(dropout): Dropout(p=0.0, inplace=False)
(classifier): Linear(in_features=312, out_features=2, bias=True)
)

# reload our model/tokenizer. Optional, only usable when in Python
# files instead of notebooks
tokenizer = BertTokenizerFast.from_pretrained(model_path)

```

Pediction 模型使用

```

# Function to make predictions
def predict_sentiment(text, model, tokenizer, device):
    # Tokenize the input text
    inputs = tokenizer(
        text,
        max_length=512,
        truncation=True,
        return_tensors="pt"
    ).to(device)

    # Get model predictions
    with torch.no_grad():
        outputs = model(*inputs)

    # Extract logits and apply softmax to get probabilities
    # logits = outputs.logits
    logits = outputs["logits"] # 取出logits

    probabilities = torch.nn.functional.softmax(logits, dim=-1)

    # Get the predicted class (0: negative, 1: positive)
    predicted_class = torch.argmax(probabilities, dim=-1).item()

    # Get the class name using id_to_label
    predicted_label = id_to_label[predicted_class]

    # Get the confidence score
    confidence = probabilities[0][predicted_class].item()

    return {
        "text": text,
        "sentiment": predicted_label,
        "confidence": round(confidence, 2),
        "probabilities": {
            id_to_label[i]: round(prob.item(), 2) for i, prob in
enumerate(probabilities[0])

```

```

    }
}

text = "今天天氣真好，我很開心"
predict_sentiment(text, model, tokenizer, device)

{'text': '今天天氣真好，我很開心',
 'sentiment': '正面',
 'confidence': 0.66,
 'probabilities': {'負面': 0.34, '正面': 0.66}}

text = "這個產品品質差，服務更糟糕"
predict_sentiment(text, model, tokenizer, device)

{'text': '這個產品品質差，服務更糟糕',
 'sentiment': '負面',
 'confidence': 0.99,
 'probabilities': {'負面': 0.99, '正面': 0.01}}

text = "這家餐廳的食物美味，環境也很舒適"
predict_sentiment(text, model, tokenizer, device)

{'text': '這家餐廳的食物美味，環境也很舒適',
 'sentiment': '正面',
 'confidence': 0.99,
 'probabilities': {'負面': 0.01, '正面': 0.99}}

```

How does it get results?

```

0 Negative
1 Positive

text = "我不高興"

inputs = tokenizer(text, padding=True, truncation=True,
max_length=512, return_tensors="pt").to(device)
inputs

{'input_ids': tensor([[ 101, 2769, 679, 7770, 5646, 102]]),
 device='cuda:0'), 'token_type_ids': tensor([[0, 0, 0, 0, 0, 0]]),
 device='cuda:0'), 'attention_mask': tensor([[1, 1, 1, 1, 1, 1]]),
 device='cuda:0')}

# perform inference to our model
outputs = model(**inputs)
outputs

```

```
SequenceClassifierOutput(loss=None, logits=tensor([[ 1.2495, -0.9947]], device='cuda:0', grad_fn=<AddmmBackward0>), hidden_states=None, attentions=None)
```

```
outputs[0]
```

```
tensor([[ 1.2495, -0.9947]], device='cuda:0', grad_fn=<AddmmBackward0>)
```

```
outputs.logits
```

```
tensor([[ 1.2495, -0.9947]], device='cuda:0', grad_fn=<AddmmBackward0>)
```

```
outputs.logits.softmax(1)
```

```
# [ 1, 0 ] --> 0  
# [ 0, 1 ] --> 1
```

```
# [ 1, 0, 0 ] --> 0  
# [ 0, 1, 0 ] --> 1  
# [ 0, 0, 1 ] --> 2
```

```
tensor([[0.9042, 0.0958]], device='cuda:0', grad_fn=<SoftmaxBackward0>)
```

```
# get output probabilities by doing softmax  
probs = outputs[0].softmax(1)  
print(probs)
```

```
tensor([[0.9042, 0.0958]], device='cuda:0', grad_fn=<SoftmaxBackward0>)
```

BERT 與 GPT 模型中的 padding 設定差異解析

為什麼 BERT 模型需要明確的 padding="max_length"

BERT 模型需要明確設定 padding="max_length" 主要有以下原因：

1. 雙向注意力機制：BERT 同時處理文本的左右兩側上下文，這種雙向處理要求所有序列必須具有一致的維度才能有效計算。
2. 並行處理需求：BERT 的自注意力層需要在批次中的所有文本上同時計算注意力分數，因此需要統一長度。

3. 位置編碼特性：BERT 的位置編碼是為固定長度序列設計的，通常最大支援 512 個 tokens。
4. 注意力遮罩機制：BERT 需要明確知道哪些是實際內容，哪些是填充部分，以免填充部分干擾注意力計算。

為什麼 GPT 模型可能不需要明確設定

GPT 模型在很多實現中可以不需要明確設定 `padding="max_length"`，原因如下：

1. 單向自迴歸特性：GPT 只從左到右處理文本，這種設計天然更適合處理變長序列。
2. 默認處理方式：很多 GPT 實現中的框架會自動處理填充問題，有更智能的默認行為。
3. 批次處理差異：GPT 相關框架可能採用不同的批次處理策略，能夠更好地處理不同長度的輸入。

效率問題分析

使用 `padding="max_length"` 不是最有效的方法，因為：

- 所有序列都被填充到最大長度（512），即使很短的句子也會佔用全部空間
- 會造成大量不必要的記憶體消耗
- 增加不必要的計算時間，特別是在處理大量短文本時

更高效的替代方案

一個更有效的做法是：

```
def tokenize_function(example):  
    return tokenizer(  
        example["text"],  
        padding=True, # 使用動態填充而非固定最大長度  
        truncation=True,  
    )
```

然後使用資料收集器來動態處理每個批次的填充：

```
from transformers import DataCollatorWithPadding  
data_collator = DataCollatorWithPadding(tokenizer=tokenizer)  
  
trainer = Trainer(  
    # 其他參數  
    data_collator=data_collator,  
)
```

這種方法只會在每個批次中填充到該批次中最長序列的長度，而不是固定填充到最大長度，能夠有效節省記憶體和計算資源，同時確保 BERT 模型的需求得到滿足。

